Engineering with Computers (2022) 38:4409–4421
https://doi.org/10.1007/s00366-022-01632-7

ORIGINAL ARTICLE

MGNet: a novel differential mesh generation method based on unsupervised neural networks

Xinhai Chen[1,2] · Tiejun Li[2] · Qian Wan[2] · Xiaoyu He[1] · Chunye Gong[2] · Yufei Pang[1] · Jie Liu[2]

Received: 13 December 2021 / Accepted: 18 February 2022 / Published online: 10 March 2022

Abstract
Mesh generation accounts for a large number of workloads in the numerical analysis. In this paper, we introduce a novel differential method MGNet for structured mesh generation. The proposed method poses the meshing task as an optimization problem. It takes boundary curves as input, employs a well-designed neural network to study the potential meshing (mapping) rules, and finally outputs the mesh with a desired number of cells. The whole process is unsupervised and does not require a priori knowledge or measured datasets. We evaluate the performance of MGNet in terms of mesh quality, network designs, robustness, and overhead on different geometries and governing equations (elliptic and hyperbolic). The experimental results prove that, in all cases, the proposed method is capable of generating acceptable meshes and achieving comparable or superior meshing performance to the traditional algebraic and differential methods. The proposed MGNet also outperforms other neural network-based solvers and enables fast mesh generation using feedforward prediction techniques.

Keywords Mesh generation · Differential method · Neural network · Unsupervised learning

1 Introduction

Meshing technology plays a vital role in a wide range of problems arising in computational science, such as aerospace, physical geography, energy engineering, and automotive engineering [1–4]. The principle of the meshing process is to divide the physical domain into mesh cells, thus providing a discrete manner to enable the desired simulation to be performed. Structured meshes are meshes whose organization of points and cells are regular and defined by a general rule: each inner vertex is topologically connected to its neighbors and referenced by the regular indices of the coordinate [5]. Due to their straightforward connectivity nature, structured meshes provide a good interpolation function in the finite-element method and are frequently used in many engineering applications for clean and accurate numerical analysis [4, 6]. The goal of the structured meshing process is to construct simplicial cells that fill the interior domain of an object within the constraints of certain mapping rules. Typically, the mapping is defined by a transformation from a regular computational mesh represented by (ξ, η) domain space into an irregular physical mesh represented by (x, y) domain space

$$x = x(\xi, \eta), \tag{1}$$

$$y = y(\xi, \eta). \tag{2}$$

In the field of automatic structured mesh generation, a great amount of work has been done. Algebraic methods use various forms of interpolation (e.g., Lagrange and Hermite transfinite interpolation) to compute the intermediate transformation $(\xi, \eta) \mapsto (x, y)$ [5]. The distribution of mesh points and lines is constrained by tangential derivatives on the physical boundary and control parameters in the transfinite interpolation formulas. Despite their simplicity, these methods can produce degenerate or overlapped cells, making

Xinhai Chen and Qian Wan have contributed equally to this work.

✉ Tiejun Li
tjli@nudt.edu.cn

Xinhai Chen
chenxinhai16@nudt.edu.cn

Jie Liu
liujie@nudt.edu.cn

[1] China Aerodynamics Research and Development Center, Mianyang 621000, China

[2] Laboratory of Software Engineering for Complex System, National University of Defense Technology, Changsha 410073, China

it difficult for meshing regions with irregular or deformed boundaries.

To address this problem, differential methods are commonly used to generate meshes for arbitrary boundaries [5]. Given the boundary values of the geometry, differential methods pose the whole mesh generation process as a boundary value problem (BVP) governed by partial differential equations (PDEs) within a framework of differential geometry [7]. During this process, a transformation between computational and physical domains is facilitated by iterative solving the underlying PDEs and propagating the known boundary points into the physical interior in a smooth manner. The steady-state solutions are the obtained structured mesh. Based on the mathematical basis of the derivation, the use of elliptic or hyperbolic systems allows for orthogonality of near-wall regions, non-propagation of boundary singularities, and nonoverlapping mesh cells. However, the conventional solving approach to obtaining valid meshes involves tedious numerical iterations [8–10]. One of the primary bottlenecks to efficient and advanced mesh generation is specifically the time it takes to solve these large-scale nonlinear systems; especially for the cases where a fast meshing process is desired, such as global re-meshing and large-scale numerical simulations in the context of real-time analysis and dynamics design [6, 11]. This motivates the study of a meshing method that is efficient with regard to the computation time and is user-friendly to allow users to generate high-quality meshes with arbitrary sizes in a short time.

Deep neural networks and their applications have drawn a special attention in many science and engineering communities involving cumbersome knowledge-based systems, such as mesh quality evaluation [12, 13], flow-field prediction [14, 15], and vortex visualization [16]. This is not the first time that researchers have attempted to apply neural networks to accelerate the meshing process. Alfonzetti et al. [17] first developed a neural network-based algorithm for unstructured mesh generation. Starting from a coarse mesh of triangles with a small number of nodes, the neural algorithm is employed to increase the number of mesh points until the user-selected value. Ahmet and Ahmet [18] introduced a 2-D mesh generation method by means of feedforward single-layer neural networks. The network takes all boundary control points as input and predicts the spatial coordinates of interior points. However, this method is very sensitive to the weight values and, therefore, requires a careful weight adjustment to ensure its accuracy. If weight values are not appropriately selected, the computed values can be outside the shape.

Zhang et al. [19] presented a mesh generation method, MeshingNet, to improve the efficiency of non-uniform mesh generation based on deep learning. They first built up an expensive training dataset by computing high-accuracy solutions on high-density uniform meshes using a traditional finite-element solver. Then, the proposed MeshingNet is trained on the dataset to learn the posteriori error features and predict the mesh density for refinement. Similar work is done by Huang et al. in [20]. They proposed a neural network-based method to learn refinement maps and predict optimal mesh densities for 2-D wind tunnels. They first convert the underlying mesh into a regular grayscale image, where the grayscale channel indicates the relative size of the cells in different regions. Then, they create a staircase U-Net architecture to learn refinement maps around a random geometry and predict an average mesh cell size per pixel.

Alexis et al. [10] provided a recent addition for mesh generation with supervised neural networks. In their method, three neural networks are used to predict the number of vertices to be inserted inside the 2-D cavity, vertex location, and vertex connectivity. Specifically, they first apply a feature transformation (scaling and rotation) to circumscribe the input physical contour in a unitary circle, which provides an invariant shape representation for the contour. Next, they take as input the boundary coordinates and target edge length and output of the number of vertices that should be inserted inside the cavity of the contour. Meanwhile, the second network is employed to predict the coordinates of the inner vertices. The obtained approximation of vertex coordinates is then fed into the third network to compute a connection table that contains values representing the probability of connecting information. Finally, the mesh is created using a triangulation algorithm according to the values of the connection table. The whole pipeline is trained on a contour dataset generated by constrained Delaunay triangulation in Gmsh [21]. Instead of inserting inner vertices one by one, the trained meshing scheme offers a more direct pathway to acquire a mesh that satisfies element size criteria, thus speeding up the meshing process procedure. However, the growth in mesh complexity (i.e., mesh size) requires more training data to attain an acceptable prediction accuracy from the neural network-based methods. The dramatic data generation overhead and preprocessing time make it difficult to apply the data-driven neural network method to large-scale meshes.

Recently, pioneering works began to explore the possibility of solving PDEs by means of unsupervised neural networks, where no pre-prepared or measured data are needed [22–24]. In these methods, the governing equations, as well as the boundary conditions, are embedded in the loss function as penalizing terms. At the same time, a neural network is employed to constrain the space of latent solutions. After suitable training, the resulting network is able to form a new class of universal function approximators that naturally encode any underlying mathematical and physical laws. The advancements of neural network-based solvers are proven to be efficient for tasks such as flow-field prediction and inversion solving, which open a new way to enable fast and accurate inference of PDE solutions.

In this paper, we consider the application of unsupervised neural networks to the differential-based meshing tasks and introduce a novel method, MGNet, for structured mesh generation. Specifically, we take a set of physical constraints (boundary curves) as input and train the MGNet to study the relationships inherent in the transformation between computational and physical domains. During this process, the governing partial differential equations play a fundamental role in the adaptation of the mesh to the given geometrical constraints. We embed the differential equation in the loss function and use it as a penalty term to guide the optimization of underlying network variables. After that, the trained MGNet can be used to generate arbitrary-size meshes by means of feedforward prediction. We conduct a series of experiments to verify the efficiency of the proposed method. The results demonstrate that MGNet is able to capture the internal features of coordinate transformations and produce valid meshes. It can also offer a computational advantage over the traditional works using algebraic or differential methods, making it a useful tool if incorporated into the mesh generation workflow.

To the best of our knowledge, we are the first to incorporate unsupervised neural networks into differential mesh generation methods. The rest of the paper is organized as follows. Section 2 provides the implementation details of the proposed neural network-based differential method for mesh generation. Section 3 presents and discusses the results in terms of robustness, mesh quality, and efficiency on different test cases. Section 4 concludes the paper and discusses the direction for future works.

2 MGNet: a novel differential mesh generation method

In this work, we pose the traditional mesh generation task as an optimization problem. Given the boundary values of the geometry and a family of governing PDEs, we assume that there is a mapping: $(\xi, \eta) \mapsto (x, y)$ that can be learned. The proposed methodology utilizes the universal function approximation ability of deep neural networks to capture the subtle input–output mapping from high-dimensional nonlinear spaces. We illustrate the overall pipeline of the proposed method in Fig. 1. The process of NN-based differential mesh generation can be divided into three steps: (1) boundary preprocessing, (2) network construction, and (3) training and prediction. Details are given in the following sections.

2.1 Boundary preprocessing

Unsupervised neural network-based mesh generation involves iterative training over a large amount of randomly sampled boundary data to satisfy specified physical boundary constraints. However, generally, boundary curves of the input geometry are represented as a free form, which means that the boundary constraints are represented with finite control points instead of whole points. This representation limits the sampling of boundary training data. Thus, to obtain sufficient boundary points, we start with fitting the boundary function using the existing control points. If the input boundary curve is given as a mathematical function, the process on step (1) does not need.

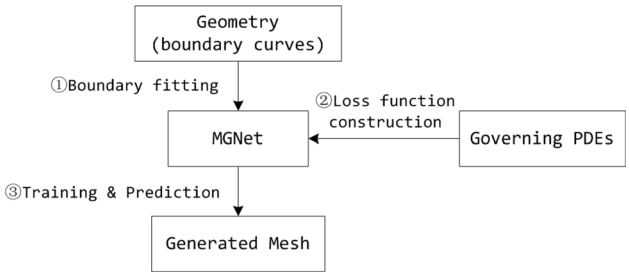

Fig. 1 The overall pipeline of the proposed method

An accurate fitting for the boundary curves is very crucial to the overall mesh output of the presented meshing method, since the resulting fitting functions serve as a reference to guide the subsequent network training. To this end, a regression analysis is opted to perform boundary fitting around control points before training. In our experiments, we compared the fitting performance and overhead of widely used regression models on the boundary curves, including polynomial linear regression, decision tree regression, K-neighbors regression, random forest regression, AdaBoost regression, and gradient boosting regression [25]. After trial-and-error tests, we finally employ a decision tree regression (DTR)-based regression model in the boundary preprocessing step to approximate the potential mapping $(\xi, \eta) \mapsto (x, y)$ on the input geometry boundary curves (see Fig. 2). This model is capable of providing continuous constraints for boundary conditions. Moreover, it enables a manner to make up boundary points to the initially given boundary curve and offers boundary training data for the subsequent training. It is worth noting that we will also use the obtained boundary fitting functions to correct the coordinates of boundary points in the prediction stage and eliminate the prediction error on the boundary curves.

2.2 Network construction

In this section, we introduce the well-designed network architecture for neural network-based differential mesh generation tasks. The core of the proposed method is to find an appropriate mapping between the computational and physical domains

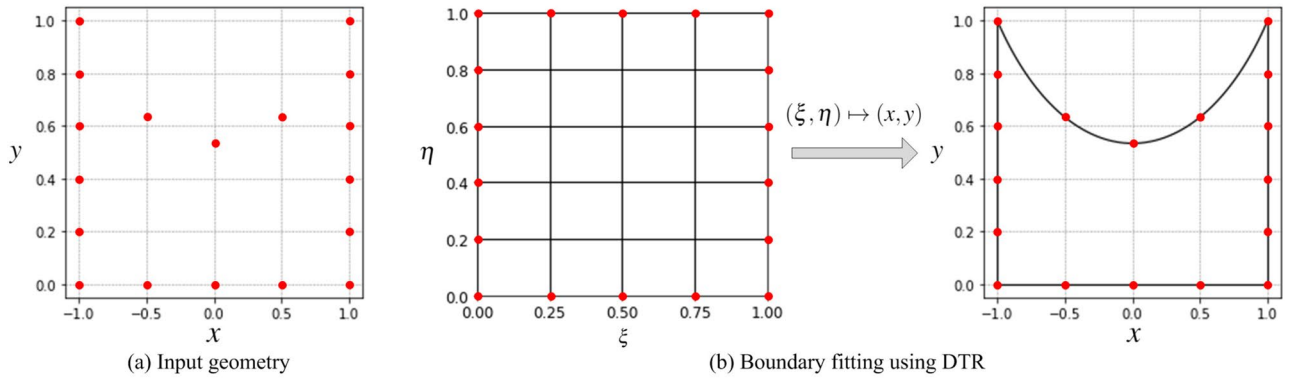

Fig. 2 Boundary preprocessing in the proposed MGNet

from the solution space of the underlying partial differential equations. Mathematically, a network model F is built to approximate the latent solution $\hat{F}$ for the desired intermediate transformation

$$\hat{F}(\xi,\eta) \approx F(\theta_{W,b},\xi,\eta), \tag{3}$$

where $\theta_{W,b}$ is a set of network parameters, including the weights and biases of the constructed network. The network approximates the mapping solution by minimizing the weighted residuals of the governing equation and boundary constrains terms, where no a priori dataset is needed. After training, the network is able to predict the target mesh distribution at any input mesh size. The meshing of points is calculated in feedforward prediction by means of deep neural networks where a burdensome overhead to solving the PDEs is not needed.

The architecture of MGNet is shown in Fig. 3. The input is two sets of training points sampled from the interior and boundaries of the computational domain. Each set of points corresponds to a sequence of 2-D points $P=\{(\xi_n,\eta_n)\}_{n=1}^{n=N}$, where N denotes the number of points.

MGNet uses two sub-networks of the same structure to predict the coordinates of the mesh points in the physical domain. Each sub-network is constituted by an input layer, an expanding layer, a series of hidden layers, and an output layer. The input layer takes training points (ξ,η) as input and forwards them to the expanding layer. The expanding layer

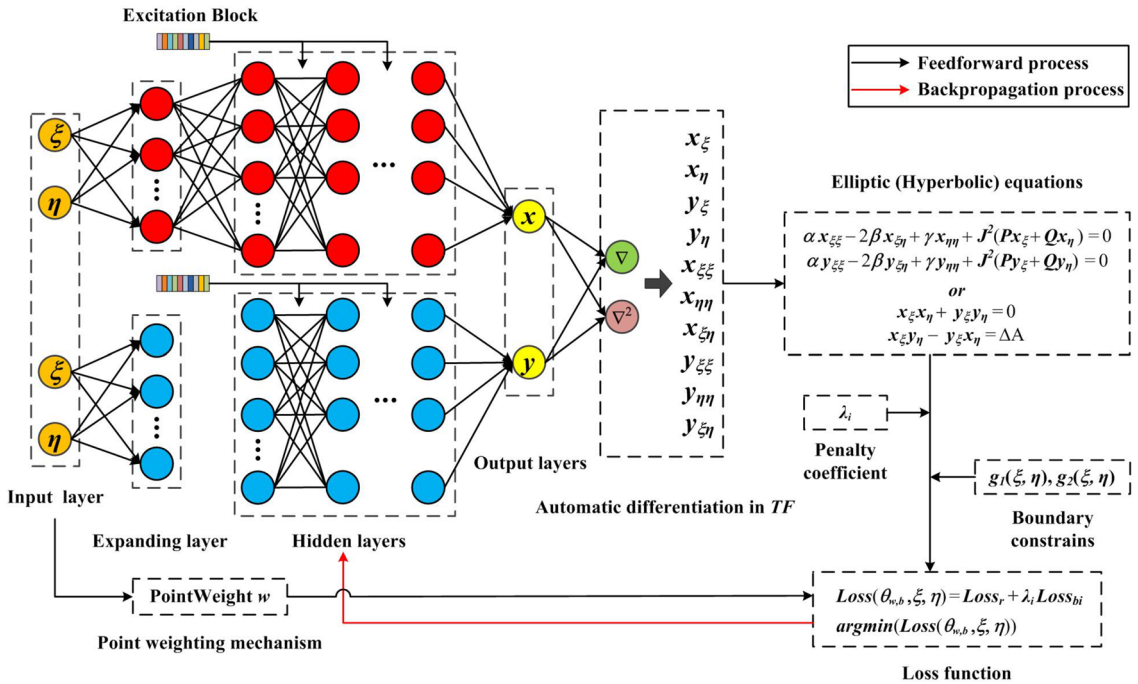

Fig. 3 The architecture of the proposed MGNet.

works as a data enhance mechanism to map the input data to a higher dimension. This enhancement is defined as

$$[(\xi,\eta)] \mapsto [\sin(\xi,\eta), \cos(\xi,\eta), (\xi,\eta)], \tag{4}$$

where $\sin(\cdot)$ and $\cos(\cdot)$ compute the sine and cosine of (ξ,η), respectively.

After expansion, a series of fully connected neurons are employed as hidden layers to extract the features of interest from the expanded point coordinates. Moreover, we introduce an excitation block in the sub-network structure to increase the prediction performance of MGNet. This is inspired by our previous work in [14], which shows a superior capacity for dealing with the unstable prediction problem. This block offers a self-gating ability for each hidden layer by giving per-channel modulation weights W_e to the output features. The nonlinear neural operation in each hidden layer is computed as

$$X_l = \sigma\left(W_l X_{l-1} + b_l\right) \cdot W_e, \tag{5}$$

where the subscript l denotes the index of the hidden layer, and W_l and b_l are the weight and bias tensor in layer l, respectively.

The last hidden layer in each sub-network passes the high-dimensional features to the output layer, which then produces one-dimensional predictions (the point coordinates in the physical domain). After that, we use automatic differentiation (AD) in Tensorflow [26] to calculate the differential operators and construct the loss function corresponding to the underlying PDEs for mesh generation. The loss function in MGNet is defined as

$$\text{Loss}(\theta_{W,b},\xi,\eta) \;=\; \text{Loss}_r + \lambda_i\, \text{Loss}_{bi}, \tag{6}$$

$$\lambda_i = \text{mean}\left(\frac{\partial \text{Loss}_r}{\partial \theta_{W,b}}\right) \Big/ \text{mean}\left(\frac{\partial \text{Loss}_{bi}}{\partial \theta_{W,b}}\right), \tag{7}$$

where Loss_r and Loss_{bi} represent the loss terms corresponding to the residual of the governing equation and the boundary functions obtained in Sect. 3.1. To further improve and accelerate training, we introduce a dynamic penalty strategy for MGNet-based training. The idea of this strategy is to apply a penalty coefficient λ_i for each boundary constraint Loss_{bi}. At this point of the process, the penalty coefficient can balance the interplay between all terms in the equation automatically and resolve the gradient pathologies problem in neural networks, thus enhancing the convergence of MGNet.

Finally, the network variables $\theta_{W,b}$ are updated and optimized through iteratively minimizing $\text{Loss}(\theta_{W,b},\xi,\eta)$ using non-convex optimization algorithms (e.g., stochastic gradient descent and quasi-Newton algorithm). The optimization process stops after converging to a local optimum. During this process, one bad configuration usually appears when the duality of boundary values exists at corner points of rectangular domain problems being considered. This duality (or discontinuity) of the corner value is introduced by fitting errors for different boundary curves. However, this type of error can cause significant problems, because once the fitting functions are fixed; they will be used as target loss terms and never have the possibility to change again. The discontinuity will keep propagating during the feedforward and backpropagation process, thus causing training instability and producing degenerate cells near the intersection regions. To this end, we apply a point weighting mechanism to overcome this issue. The mechanism uses a parabolic function to assign different weights to the sampled points on the boundary, where the midpoint has a weight of 1, and the endpoints have a weight of 0.

Algorithm 1 The training procedure of the proposed MGNet

Require: ξ and η

Ensure: A set of (sub)optimal network parameters $\theta_{W,b}$

1: Select training points $P_r : (\xi_n^r,\ \eta_n^r)_{n=1}^{n=N_r}$, $P_b : (\xi_n^b,\ \eta_n^b)_{n=1}^{n=N_b}$, randomly from the computational domain;
2: Fit the boundaries using Decision Tree Regression (DTR);
3: Calculate point weights w;
4: Construct the network model MGNet $F(\theta_{W,b},\xi,\eta)$;
5: Construct the loss function $Loss(\theta_{W,b},\xi,\eta)$ in MGNet;
6: Minimize the loss function via non-convex optimization algorithms and update the network parameters.

Algorithm 1 shows the detailed training procedure of the proposed NN-based differential mesh generation method. We first randomly select the training point P_r and P_b from the computational domain (step 1). In step 2, we employ the decision tree regression model to fit the physical boundaries. Steps 3–6 correspond to the training of the MGNet. We input the obtained boundary functions as well as the point coordinates and calculate the point weights w for each input sample (step 3). Then, we construct the network architecture (step 4) and the loss function corresponding to the governing differential equation (step 5). We optimize the parameter θ by applying forward- and back-propagating. During this process, penalty coefficients are updated every ten epochs to avoid the imbalance contribution of different loss terms. The loss function is minimized via non-convex optimization algorithms such as gradient descent and quasi-Newton method (step 6). This iterative process stops when the number of training epoch reaches the user-specified value. In this way, we obtain a neural network-based mesh generator that implies the desired mapping of mesh points from the computational domain to the physical domain.

Beyond its novelty, the proposed MGNet can be straightforward and efficient. Compared with the traditional numerical-based meshing methods, the entire meshing process of MGNet is extremely fast, since it only involves a few matrix multiplications during the feedforward prediction procedure. After suitable offline training, MGNet is able to produce arbitrary-size meshes for the input geometry at the first attempt. Specifically, the trained MGNet can be viewed as a regression model that captures the mapping relationship between the computational and physical domains. This regression model takes point coordinates (ξ, η) in the computational domain as inputs and outputs the corresponding coordinates (x, y) in the physical domain. Therefore, we can easily generate arbitrary-size meshes by adjusting the number of input points. For example, we can generate a coarse mesh by sparsely sampling the boundary and interior points in the computational domain or further refine the mesh by increasing the number of sampled points. We can also incorporate it into the current mesh generation processes or combine it with mesh optimization algorithms to enable intelligent, high-quality mesh generation and improve efficiency.

2.3 Training and prediction

We now introduce the training and hyperparameter setting of the proposed MGNet. In our experiments, we do not consider a very deep network structure. Although a large-scale network model trained on GPUs can yield better results, we restrict our network size to be handled at the CPU level to demonstrate the practicality of the proposed methodology. Consequently, we contract a four-hidden-layer MGNet with 100 neural units per layer to learn the meshing rules. The tanh function is employed in each layer except the last one. This activation function is defined as

$$\sigma(x) = \frac{\sinh x}{\cosh x} = \frac{e^x - e^{-x}}{e^x + e^{-x}}. \tag{8}$$

During each epoch, we randomly select 100 interior points and 400 boundary points from the computational domain for training. We jointly use two optimizers, Adam and L-BFGS-B [27], in MGNet to optimize the adjustable parameters. Specifically, we first apply the Adam optimizer for 15,000 epochs of stochastic gradient descent training, where the initial learning rate is set to 1e−3 and decays by 0.9 per 1000 epochs. We then employ the limited-memory quasi-Newton optimizer L-BFGS-B for further bound-constrained optimization. The procedure aims to avoid getting trapped in a local optimum and produce high-quality meshing results. For all test cases, the implementation is done using Tensorflow, and the training is conducted on Intel Xeon Silver 4116 CPUs.

3 Experimental results

In this section, we present the results of MGNet governed by different partial differential equations. In each case, we compare the quality of the generated mesh with that generated using traditional methods such as the algebraic method and differential (PDE) method. For the algebraic method, we use the formulas of Lagrange transfinite interpolation, while the central difference is used in the differential method. In our experiments, the number of mesh nodes distributed on each boundary curve is set to 100. To evaluate the performance of MGNet in more detail, we also compare it with widely used NN-based PDE solvers and study the choice of hyperparameters and the efficiency of the proposed method.

3.1 Laplace equation

In the first test case, we employ two geometric examples to investigate the capability of the neural network-based mesh generator and to give some practical settings of MGNet governed by the Laplace equation. The Laplace equation is of the form

$$\begin{aligned}
&\alpha x_{\xi\xi} - 2\beta x_{\xi\eta} + \gamma x_{\eta\eta} = 0, \quad (\xi, \eta) \in \Omega,\\
&\alpha y_{\xi\xi} - 2\beta y_{\xi\eta} + \gamma y_{\eta\eta} = 0, \quad (\xi, \eta) \in \Omega,\\
&x = g_1(\xi, \eta), \quad (\xi, \eta) \in \partial\Omega,\\
&y = g_2(\xi, \eta), \quad (\xi, \eta) \in \partial\Omega,
\end{aligned} \tag{9}$$

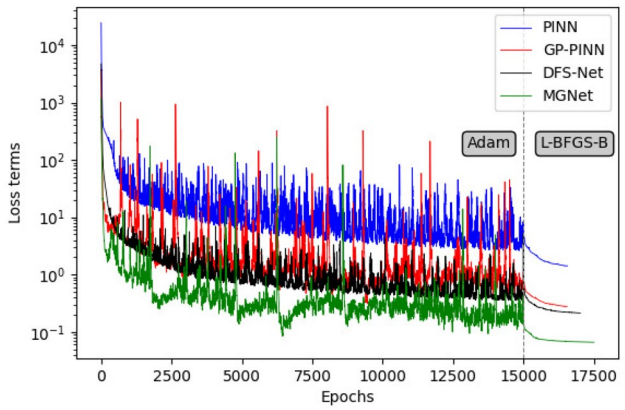

Fig. 4 The convergence of different neural network-based solvers on the mesh generation task

where

$$\alpha = x_\eta^2 + y_\eta^2, \quad \beta = x_\xi x_\eta + y_\xi x_\eta, \quad \gamma = x_\xi^2 + y_\xi^2. \tag{10}$$

To approximate the implied mapping from the parametric coordinates to the physical coordinates, we formulate the loss function as follows:

$$\begin{aligned} Loss(\theta_{W,b}, x, y) = \frac{1}{N_r}\sum_{n=1}^{N_r} w_n \cdot (\|\alpha x_{\xi\xi} - 2\beta x_{\xi\eta} + \gamma x_{\eta\eta}\|^2 \\ + \|\alpha y_{\xi\xi} - 2\beta y_{\xi\eta} + \gamma y_{\eta\eta}\|^2) \\ + \frac{1}{N_b}\sum_{n=1}^{N_b} w_n \cdot (\|x - g_1(\xi,\eta)\|^2 + \|y - g_2(\xi,\eta)\|^2). \end{aligned} \tag{11}$$

During training, we use both the Adam and L-BFGS-B optimizers to find the optimal network parameters that minimize the loss function in Eq. 11. To verify the effectiveness of the well-designed network architecture and the introduced weighting tricks, we first compare the performance of MGNet with other widely used network designs on meshing tasks. The networks are PINN in Ref. [28], GP-PINN in Ref. [29], and DFS-Net in Ref. [14]. For a fair comparison, we use the same hyperparameter and network scale settings. The loss convergence of different network models is depicted in Fig. 4 and the corresponding visualization results are plotted in Fig. 5. The results show that existing neural networks suffer from poor convergence, and the meshes generated by these networks tend to be of poor quality.

We can see that the MGNet, designed for neural network-based meshing tasks, outperforms other commonly used deep neural networks. The loss value of MGNet is approximately two orders of magnitude lower than those of PINN and one order of magnitude lower than those of GP-PINN and DFS-Net. The experimental results also prove that the combined use of the two optimizers in MGNet is suitable for the neural network-based meshing tasks. In the first phase, the loss value decreases as the number of training epochs increases, wherein the point weighting mechanism and dynamic penalty strategy automatically adjust the optimization direction to ensure convergence of the training. After the first stage of convergence, L-BFGS-B, a limited-memory quasi-Newton optimizer, is employed to finetune the results and get rid of the local optima. Finally, the MGNet

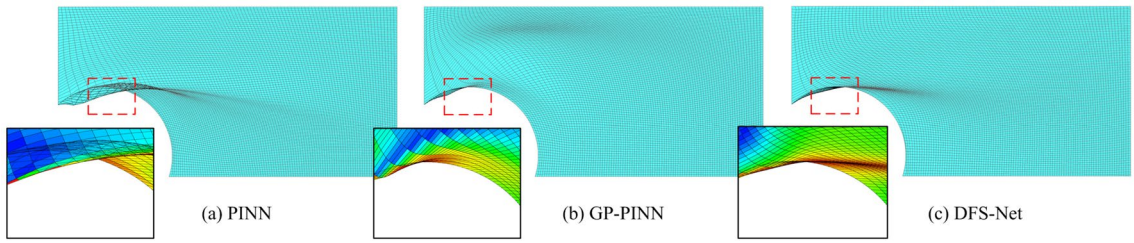

Fig. 5 Visualization results of different deep neural networks on example 1

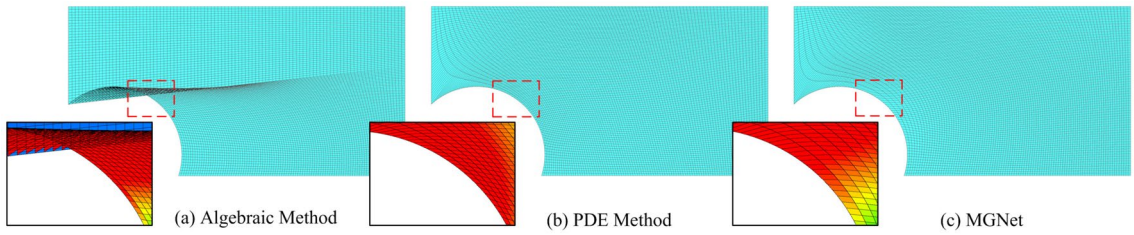

Fig. 6 Visualization results of different mesh generation methods on example 1. **a** Algebraic method: average included angle = 111.76; **b** PDE method: average included angle = 113.52; **c** MGNet: average included angle = 110.55

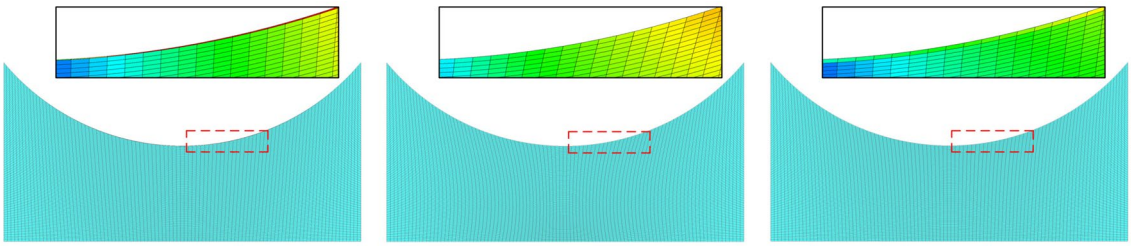

Fig. 7 Visualization results of different mesh generation methods on example 2. **a** Algebraic method: average included angle = 106.09; **b** PDE method: average included angle = 107.22; **c** MGNet: average included angle = 105.59

converges after 15,000 epochs of Adam training and 2513 epochs of L-BFGS-B training.

Figures 6c and 7c visualize the meshing results of the proposed MGNet governed by the Laplace equation on two different examples. For the sake of comparison, the results of algebraic and differential methods are also shown in Figs. 6 and 7, respectively. Prior studies acknowledge something that mesh engineers inherently know: that skewed mesh cells with poor orthogonality can negatively affect the entire simulation and cause large computational errors, and thus, the lines of the mesh should be as orthogonal as possible [12, 13]. Generally, a mesh with good orthogonality has better convergence and faster convergence speed. Therefore, in each test example, we employ a skewness metric, the average included angle, to evaluate and compare the quality of the generated meshes. We can see that algebraic methods, which use transfinite interpolation, are not always able to produce an acceptable mesh. The computational mesh obtained by the algebraic method suffers from cell degeneration near the upper boundary (see Fig. 7a). At worst, in regions of complicated shape, the mesh cells can overlap and even cross the boundary (see Fig. 6a). Compared with the algebraic method, the differential and neural network-based methods do not produce cell overlap and allow for the full smoothness of mesh nodes within the physical domain. Among them, the mesh generated by MGNet also achieves better orthogonality. The average included angles for the first and second examples are 105.59 and 110.55, respectively, both of which are smaller than the meshes generated by the traditional methods.

3.2 Poisson equation

The systems of Poisson equations have been widely studied in structured mesh generation to intersect the boundary in a nearly orthogonal fashion. In this case, we train the MGNet using the Poisson equation, which is defined by

$$\begin{aligned} &\alpha x_{\xi\xi} + \gamma x_{\eta\eta} = -J^2 x_\xi P - J^2 x_\eta Q, \\ &\alpha y_{\xi\xi} + \gamma y_{\eta\eta} = -J^2 y_\xi P - J^2 y_\eta Q, \\ &x = g_1(\xi,\eta), \quad (\xi,\eta) \in \partial\Omega, \\ &y = g_2(\xi,\eta), \quad (\xi,\eta) \in \partial\Omega, \end{aligned} \tag{12}$$

where

$$\begin{aligned} &\alpha = x_\eta^2 + y_\eta^2, \quad \gamma = x_\xi^2 + y_\xi^2, \\ &J = x_\xi y_\eta - x_\eta y_\xi, \quad P = \phi/\gamma, \quad Q = \psi/\alpha, \\ &\phi = -\frac{x_\xi x_{\xi\xi} + y_\xi y_{\xi\xi}}{\gamma} + \frac{x_\xi x_{\eta\eta} + y_\xi y_{\eta\eta}}{\alpha}, \\ &\psi = -\frac{x_\eta x_{\eta\eta} + y_\eta y_{\eta\eta}}{\alpha} + \frac{x_\eta x_{\xi\xi} + y_\eta y_{\xi\xi}}{\gamma}. \end{aligned} \tag{13}$$

The corresponding loss function is

$$\begin{aligned} \mathrm{Loss}(\theta_{W,b}, x, y) \;=\; & \frac{1}{N_r}\sum_{n=1}^{N_r} w_n \cdot (\|\alpha x_{\xi\xi} + \gamma x_{\eta\eta} + J^2 x_\xi P + J^2 x_\eta Q\|^2 \\ & + \|\alpha y_{\xi\xi} + \gamma y_{\eta\eta} + J^2 y_\xi P + J^2 y_\eta Q\|^2) \\ & + \frac{1}{N_b}\sum_{n=1}^{N_b} w_n \cdot (\|x - g_1(\xi,\eta)\|^2 + \|y - g_2(\xi,\eta)\|^2). \end{aligned} \tag{14}$$

We use three examples to evaluate the meshing performance of the proposed method. The results in Figs. 8, 9, and 10 demonstrate that MGNet can help resolve degenerated cells. It can generate smooth and orthogonal meshes and achieve comparable or superior meshing performance to the traditional methods.

From Fig. 8, we can see that the algebraic method tends to generate poorly shaped cells (slivers) in the unsmooth near-wall region, which can lead to inaccurate or non-convergence results in the numerical simulation. In contrast, MGNet can effectively improve the orthogonality of the mesh cells near the boundary. The average interior

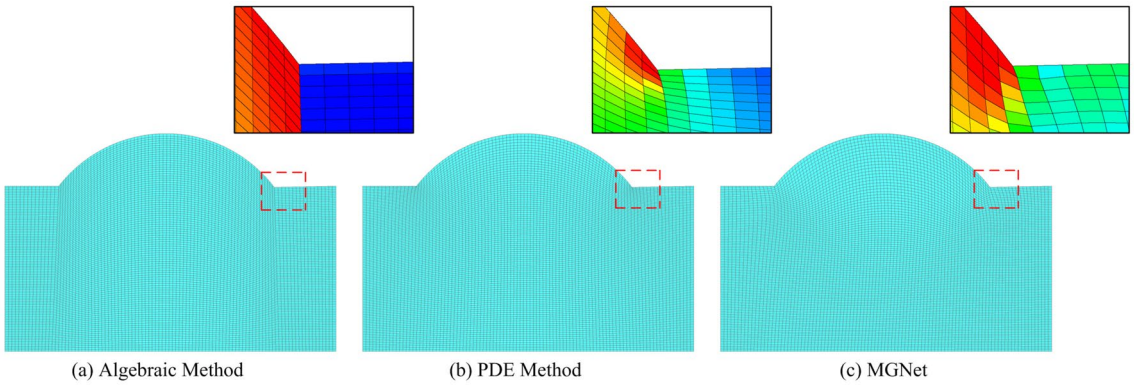

Fig. 8 Visualization results of different mesh generation methods on example 3. **a** Algebraic method: average included angle =100.42; **b** PDE method: average included angle = 98.21; **c** MGNet: average included angle = 95.91

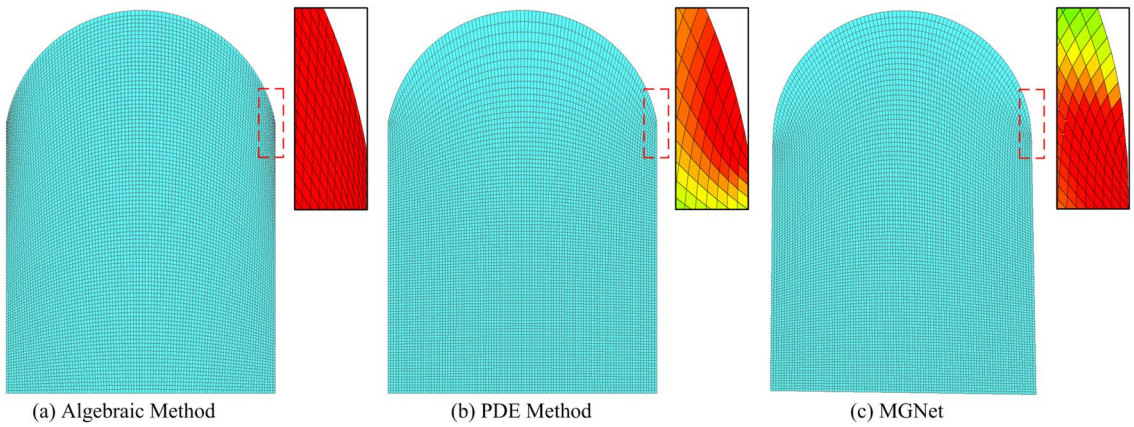

Fig. 9 Visualization results of different mesh generation methods on example 4. **a** Algebraic method: average included angle =110.28; **b** PDE method: average included angle =96.27; **c** MGNet: average included angle =97.30

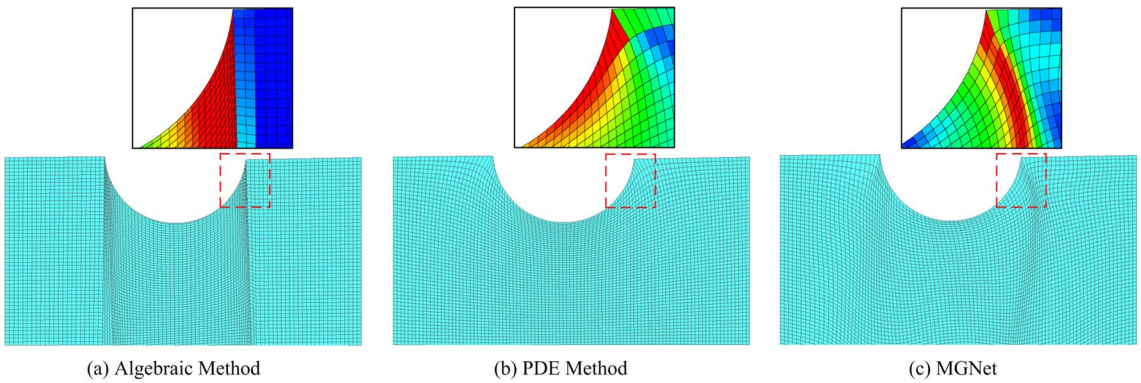

Fig. 10 Visualization results of different mesh generation methods on example 5. **a** Algebraic method: average included angle =103.81; **b** PDE method: average included angle =98.49; **c** MGNet: average included angle =98.70

angle of the MGNet-based mesh is 95.91°, which is lower than that of the mesh generated by the algebraic method (100.42) and the mesh generated by the differential method (98.21). As for Figs. 9 and 10, the mesh generated by MGNet has an average angle of 97.3 and 98.7, respectively. The metric results are slightly larger than

Table 1 The effect of different learning rates on the proposed MGNet

Learning rate	1.0E+00	1.0E−01	1.0E−02
Loss	nan	2.92E+02	4.30E+00
Learning rate	1.0E−03	1.0E−04	1.0E−05
Loss	**2.10E−02**	6.41E−02	5.21E−01

*The best performance is indicated in bold

Table 3 The effect of different optimizers on the proposed MGNet

Optimizer	AdaDelta	AdaGrad	Adam	Momentum
Loss	5.92E−02	3.49E+00	**2.10E−02**	nan
Optimizer	NAG	RMS	SGD	
Loss	7.24E+00	7.79E−02	nan	

*The best performance is indicated in bold

that generated by the differential equations but better than that of the algebraic method.

Tuning training configurations and hyperparameters is an essential process for network design in deep learning methods. To establish a training methodology for neural network-based mesh generation tasks, we analyze the effects of learning rates, activation function, and optimizers on the meshing performance.

Learning rate is one of the most crucial parameters for network training that controls the step size of the gradient descent. Table 1 summarizes the loss values for different learning rates. We can learn that an overly large learning rate might overshoot and prevent converging, while a small step size may get stuck in local minima, thus providing suboptimal solutions. In our cases, MGNet achieves the best performance when the initial learning rate is 0.001.

The activation function provides a nonlinear transformation to the output of each hidden layer, making it possible for neurons to approximate complex meshing patterns. We compare the performance of eleven different activation functions in TensorFlow [30] for our task. As shown in Table 2, elu and selu fail to converge during training and yield an invalid divergent result (*nan*). Functions like sin and swish give a reasonable performance on the meshing task. However, it is clear that the nonlinear transformation performed using tanh is more effective. It outperforms other activation functions and achieves the loss value of 2.10E−02.

We also include performance when MGNet is trained on different optimizers [26]. The experimental results of five widely used optimizers are shown in Table 3. We can see that optimizers including Adam, RMS, and AdaDelta are suitable for minimizing the loss function in MGNet training, because these optimizers are able to converge very quickly. The other optimizers, on the other hand, tend to get stuck in saddle points and yield poor quality meshes.

3.3 Hyperbolic equation

For meshing geometries with arbitrary boundaries, differential methods based on the solution of hyperbolic equations are commonly used. In the last case, we evaluate the meshing performance of MGNet governed by the hyperbolic equation. The equation we used is defined as

$$\begin{aligned} &x_\xi x_\eta + y_\xi y_\eta = 0, \quad (\xi,\eta) \in \Omega, \\ &x_\xi y_\eta - y_\xi x_\eta = \Delta A, \quad (\xi,\eta) \in \Omega, \end{aligned} \tag{15}$$

$$\begin{aligned} &x = g_1(\xi,\eta), \quad (\xi,\eta) \in \partial\Omega, \\ &y = g_2(\xi,\eta), \quad (\xi,\eta) \in \partial\Omega, \end{aligned} \tag{16}$$

where ΔA denotes the reference area of mesh cells. Similarly, we construct the hyperbolic equation-based loss function

$$\begin{aligned} \text{Loss}(\theta_{W,b}, x, y) \; = \; & \frac{1}{N_r}\sum_{n=1}^{N_r} w_n \cdot (\|x_\xi x_\eta + y_\xi y_\eta\|^2 \\ & + \|x_\xi y_\eta - y_\xi x_\eta - \Delta A\|^2) \\ & + \frac{1}{N_b}\sum_{n=1}^{N_b} w_n \cdot (\|x - g_1(\xi,\eta)\|^2 \\ & + \|y - g_2(\xi,\eta)\|^2). \end{aligned} \tag{17}$$

We conduct experiments on two different test examples: a hexagon and a 2-D airfoil. Figures 11 and 12 depict the meshing results on these two geometries. The results demonstrate again that the intermediate transformation for generating meshes on a physical geometry can be found by the proposed neural network-based differential mesh generation method.

Table 2 The effect of different activation functions on the proposed MGNet

σ	cos	elu	leaky	relu	relu6	selu
Loss	1.63E+02	nan	4.97E+01	5.87E+01	6.47E+00	nan
σ	log-sigmoid	sigmoid	sin	swish	tanh	
Loss	8.71E+02	1.12E+01	3.51E−01	9.75E−01	**2.10E−02**	

*The best performance is indicated in bold

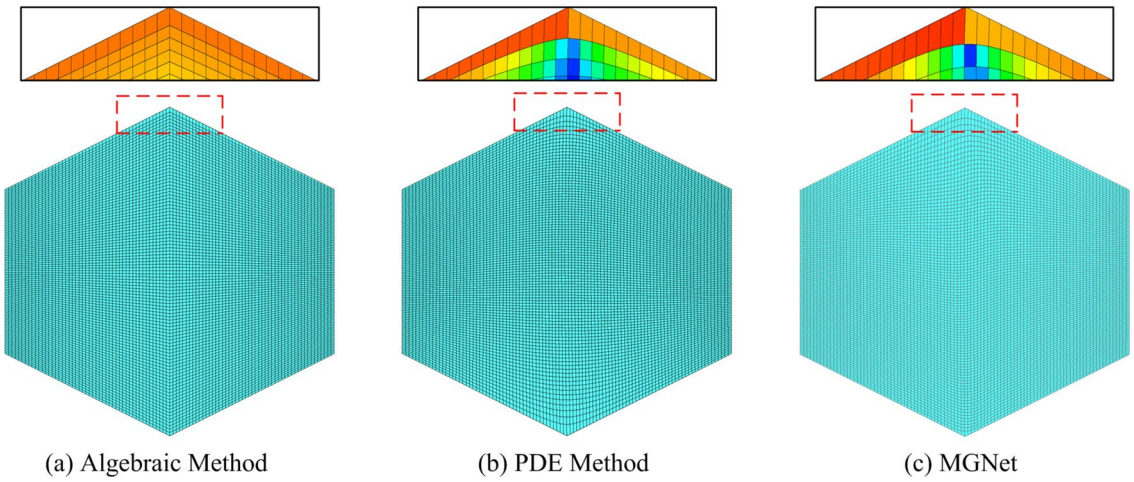

Fig. 11 Visualization results of different mesh generation methods on example 6. **a** Algebraic method: average included angle =103.92; **b** PDE method: average included angle =100.41; **c** MGNet: average included angle = 100.30

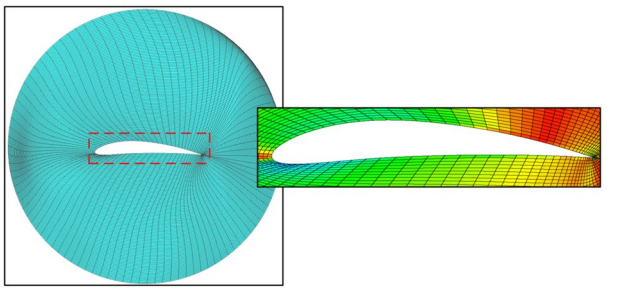

Fig. 12 Visualization results of different mesh generation methods on a 2-D airfoil

To estimate the efficiency of our process, we perform a study on the meshing overhead of the proposed MGNet. Specifically, we purposely generate meshes with an increasing mesh size on the same geometry and measure the meshing overhead of MGNet. We then compare it to the traditional methods. For the PDE method, we use the algebraic approach as an initial approximation to start the iterative process of an elliptic grid solver and record the overhead of its ten iterations. All the methods are implemented in Python and tested on a single CPU.

Table 4 lists the wall time required for mesh sizes ranging from 100×100 to 3200×3200. We can see that the mesh generation process in MGNet is very efficient. The whole process is fast and does not exceed 140 s, even for meshes with 10 million cells. Figure 13 plots the results and shows that the meshing overhead of algebraic and MGNet is linear with the increase in mesh sizes. In contrast, the wall time of the differential method increases sharply with the number of cells to be generated. In the worst extreme, the pure differential method (where the initial point coordinates are initialized to 0) can suffer from a much larger iteration overhead, usually 5–100 times the number of iterations applying algebraic-based initialization.

It is also worth noting that the mapping (prediction) process of mesh points in MGNet is data-independent. Therefore, the proposed method can be easily scaled across multiple CPUs to further reduce the overhead (e.g., using region decomposition for parallel prediction). At this point, the meshes generated via MGNet may be used in a variety of

Table 4 A comparison of meshing overhead for different sizes of meshes

Mesh size	100^2	200^2	400^2	800^2	1600^2	3200^2
Algebraic method	0.18	0.80	3.22	13.14	48.51	151.60
PDE method	2.70	7.44	25.87	93.34	367.51	1532.66
MGNet	3.59	4.82	6.87	12.06	37.69	135.07

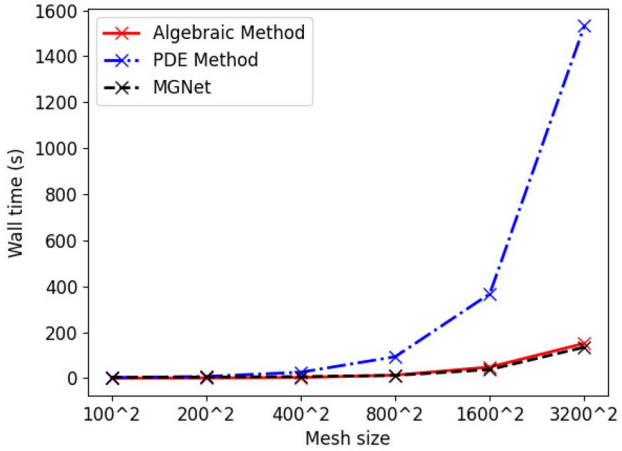

Fig. 13 The meshing overhead (wall time) of different mesh generation methods.

ways. After suitable offline training, the proposed method provides a fast online prediction to obtain a high-quality mesh with an arbitrary number of cells. We can also view MGNet as a means to generate an improved initial mesh within a traditional mesh adaptivity workflow. The quality of the obtained mesh can be further optimized using improvement heuristics such as sliver exudation, local re-meshing, and topological operations methods.

Overall, in this paper, we have proposed a novel differential mesh generation method based on unsupervised neural networks. The approach is designed for general PDEs with a range of boundaries and problem parameters. We have implemented a two-dimensional prototype and validated it on different geometries. The above tests have shown the potential of the technique in approximating the nonlinear mapping (transformation) between the computational and physical domains. The trained MGNet is capable of generating smooth meshes and achieving comparable or better meshing results than the traditional methods. Most significantly, MGNet avoids the expense of tedious numerical iterations. The meshing process involves only a few matrix multiplications and is proved to be cost-effective on large-scale meshes.

Despite the advantages and the simplicity of its use, there are two main limitations to our proposed method. The first is the boundary error. Due to the inevitable existence of feedforward and backward propagation errors, the proposed method may not guarantee a precise boundary point prediction result that exactly matches the input boundary curves. Therefore, in the current work, we only use MGNet to predict the interior mesh points. The second limitation is that the PDE equations used to construct the loss function must be continuous. This leads to the fact that some equations containing discrete source terms (e.g., the discontinuous source terms developed in Ref. [6]) cannot be applied to neural network-based methods, which limits the generalizability of MGNet.

4 Conclusion

Despite the variety of approaches in the use of deep learning methods for numerical analysis, applying deep neural networks in the field of mesh generation remains an open area with great potential. Here, we propose a neural network-based differential method to reduce the potential overhead of traditional structured mesh generation tasks. In our method, the input is a set of boundary curves, and the output is the desired mesh. The accuracy of the process is evaluated by comparing the mesh generated by the neural networks with that generated by the traditional methods and other deep learning methods. The results show that MGNet is able to achieve good performances regarding mesh quality and computation time.

In future work, we will focus on studying the implicitly encoded meshing features in the current MGNet and calibrating the model to more complex tasks. We are also considering of coupling the neural network-based methods with conformal transformation, variational method, or dynamic mesh models to improve the efficiency of the different techniques.

Acknowledgements This research work was supported in part by the National Key Research and Development Program of China (2017YFB0202104, 2018YFB0204301).

Declarations

Conflict of interest The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Chang-Hoi A, Sang-Soo L, Hyuek-Jae L, Soo-Young L (1991) A self-organizing neural network approach for automatic mesh generation. IEEE Trans Magn 27(5):4201–4204. https://doi.org/10.1109/20.105028
2. Chen X, Liu J, Li S, Xie P, Chi L, Wang Q (2018) Tamm: a new topology-aware mapping method for parallel applications on the tianhe-2a supercomputer. In: Vaidya J, Li J (eds) Algorithms and architectures for parallel processing. Springer, Cham, pp 242–256
3. Karman SL, Anderson WK, Sahasrabudhe M (2006) Mesh generation using unstructured computational meshes and elliptic partial differential equation smoothing. AIAA J 44(6):1277–1286. https://doi.org/10.2514/1.15929
4. Pochet A, Celes W, Lopes H, Gattass M (2017) A new quadtree-based approach for automatic quadrilateral mesh generation. Eng Comput 33(2):275–292. https://doi.org/10.1007/s00366-016-0471-0

5. Liseikin VD (2004) A computational differential geometry approach to grid generation. Springer, Berlin
6. Lu F, Qi L, Jiang X, Liu G, Liu Y, Chen B, Pang Y, Hu X (2020) Nnw-gridstar: interactive structured mesh generation software for aircrafts. Adv Eng Softw 145:102803. https://doi.org/10.1016/j.advengsoft.2020.102803
7. Shragge J (2006) Differential mesh generation. SEG Tech Progr Expand Abstr 10(1190/1):2369975
8. Haynes RD, Howse AJM (2015) Alternating Schwarz methods for partial differential equation-based mesh generation. Int J Comput Math 92(2):349–376. https://doi.org/10.1080/00207160.2014.891733
9. Brink AR, Najera-Flores DA, Martinez C (2020) The neural network collocation method for solving partial differential equations. Neural Comput Appl. https://doi.org/10.1016/j.jcp.2021.110364
10. Papagiannopoulos A, Clausen P, Avellan F (2021) How to teach neural networks to mesh: application on 2-d simplicial contours. Neural Netw 136:152–179. https://doi.org/10.1016/j.neunet.2020.12.019
11. Lu X, Tian B (2010) Research on mesh generation in the finite element numerical analysis based on radial basis function neural network. In: 2010 international conference on computer application and system modeling (ICCASM), vol 11, pp 157–160. 10.1109/ICCASM.2010.5623240
12. Chen X, Liu J, Gong C, Li S, Pang Y, Chen B (2021) Mve-net: an automatic 3-d structured mesh validity evaluation framework using deep neural networks. Comput Aided Des 141:103104. https://doi.org/10.1016/j.cad.2021.103104
13. Chen X, Liu J, Pang Y, Chen J, Chi L, Gong C (2020) Developing a new mesh quality evaluation method based on convolutional neural network. Eng Appl Comput Fluid Mech 14(1):391–400. https://doi.org/10.1080/19942060.2020.1720820
14. Chen X, Chen R, Wan Q, Xu R, Liu J (2021) An improved data-free surrogate model for solving partial differential equations using deep neural networks. Sci Rep. https://doi.org/10.1038/s41598-021-99037-x
15. Jin X, Cai S, Li H, Karniadakis GE (2021) NSFnets (Navier–Stokes flow nets): physics-informed neural networks for the incompressible Navier–Stokes equations. J Comput Phys 426:109951. https://doi.org/10.1016/j.jcp.2020.109951
16. Liang D, Wang Y, Chen C, Liu Y, Wang F (2020) A clustering-based approach to vortex extraction. J Vis. https://doi.org/10.1007/s12650-020-00636-z
17. Alfonzetti S, Coco S, Cavalieri S, Malgeri M (1996) Automatic mesh generation by the let-it-grow neural network. IEEE Trans Magn 32(3):1349–1352. https://doi.org/10.1109/20.497496
18. Ahmet Ç, Ahmet A (2002) Neural networks based mesh generation method in 2-d. In: Shafazand H, Tjoa AM (eds) EurAsia-ICT 2002: information and communication technology. Springer, Berlin, Heidelberg, pp 395–401
19. Zhang Z, Wang Y, Jimack PK, Wang H (2020) Meshingnet: A new mesh generation method based on deep learning. In: Krzhizhanovskaya VV, Závodszky G, Lees MH, Dongarra JJ, Sloot PMA, Brissos S, Teixeira J (eds) Computational Science—ICCS 2020. Springer, Cham, pp 186–198
20. Huang K, Krügener M, Brown A, Menhorn F, Bungartz H-J, Hartmann D (2021) Machine learning-based optimal mesh generation in computational fluid dynamics .https://arxiv.org/abs/2102.12923
21. Geuzaine C, Remacle J.-F(2009) Gmsh: a three-dimensional finite element mesh generator with built-in pre-and post-processing facilities.International Journal for Numerical Methods in Engineering 79(11), 1309–1331
22. Raissi M, Perdikaris P, Karniadakis GE (2019) Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. J Comput Phys 378:686–707. https://doi.org/10.1016/j.jcp.2018.10.045
23. Dwivedi V, Srinivasan B (2020) Physics informed extreme learning machine (PIELM)—a rapid method for the numerical solution of partial differential equations. Neurocomputing 391:96–118. https://doi.org/10.1016/j.neucom.2019.12.099
24. Arthurs CJ, King AP (2021) Active training of physics-informed neural networks to aggregate and interpolate parametric solutions to the Navier-Stokes equations. J Comput Phys. https://doi.org/10.1016/j.jcp.2021.110364
25. Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V, Vanderplas J, Passos A, Cournapeau D, Brucher M, Perrot M, Duchesnay E, Louppe G (2012) Scikit-learn: machine learning in python. J Mach Learn Res 12:2825–2830
26. Abadi M, Barham P, Chen J, Chen Z, Davis A, Dean J, Devin M, Ghemawat S, Irving G, Isard M, Kudlur M, Levenberg J, Monga R, Moore S, Murray D, Steiner B, Tucker P, Vasudevan V, Warden P, Zhang X (2016) TensorFlow: a system for large-scale machine learning. In: Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation.OSDI'16, pp. 265–283. USENIX Association, USA
27. Morales J, Nocedal J (2011) Remark on algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound constrained optimization. ACM Trans Math Softw 38:7. https://doi.org/10.1145/2049662.2049669
28. Raissi M, Yazdani A, Karniadakis GE (2020) Hidden fluid mechanics: learning velocity and pressure fields from flow visualizations. Science 367(6481):1026–1030. https://doi.org/10.1126/science.aaw4741
29. Wang S, Teng Y, Perdikaris P(2020) Understanding and mitigating gradient pathologies in physics-informed neural networks.https://arxiv.org/abs/2001.04536
30. Ramachandran P, Zoph B, Le QV (2017) Searching for activation functions. https://arxiv.org/abs/1710.05941